AI TRAINING – NUMPY ASSIGNMENT

Arrays, Dimensions, Indexing, Slicing & Vectorized Operations

Learn → Create Arrays → Access Data → Process Efficiently

1. What You Will Learn

- Create NumPy arrays from Python lists.
- Understand 1D, 2D, and 3D array dimensions and the shape attribute.
- Access array elements using positive and negative indexing.
- Select portions of arrays using slicing.
- Perform vectorized arithmetic and comparison operations without writing loops for each element.
- Complete a small marks-analysis task using NumPy.

2. Student Deliverable – Keep It Simple

Create one Python script named: numpy_assignment_YourName.py

Use NumPy (import numpy as np). Keep the code readable and divide the main task into functions where useful.

- Create and inspect arrays.
- Demonstrate dimensions and shape.
- Use indexing and slicing to access data.
- Apply vectorized operations.
- Print clear results and test your code.

3. Part A – Create NumPy Arrays

Import NumPy and complete these tasks:

1. Create a 1D array from [10, 20, 30, 40, 50].

```
import numpy as np
arr=np.array([10, 20, 30, 40, 50])
print(arr)
```

2. Create a 2D array from [[10, 20, 30], [40, 50, 60]].

```
import numpy as np
arr=np.array([[10, 20, 30], [40, 50, 60]])
print(arr)
```

3. Create a 3D array using the values 1 through 8, arranged into two 2×2 blocks.

Answer:

```
import numpy as np
arr3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(arr3)
```

4. Print each array and identify its data type using dtype.

Answer:

```
print(arr.dtype)
print(arr2.dtype)
print(arr3.dtype)
```

Create an array of five zeros and an array of five ones.

Answer:

```
zeros = np.zeros(5)
ones = np.ones(5)
print(zeros)
print(ones)
```

4. Part B – Dimensions and Shape

For each array created in Part A:

5. Print ndim to identify the number of dimensions.

```
import numpy as np
arr=np.array([[10, 20, 30], [40, 50, 60]])
print(arr.ndim)
```

output:2

6. Print shape to show the size along each dimension.

Corrected code:

```
arr = np.array([[10, 20, 30], [40, 50, 60]])
print(arr.shape)  # Output: (2, 3)
```

```
import numpy as np
arr=np.array([[10, 20, 30], [40, 50, 60]])
print(arr.size)
```

7. Print size to count the total number of elements.

```
import numpy as np
arr=np.array([[10, 20, 30], [40, 50, 60]])
print(arr.size)
```

8. In your own words, explain the difference between ndim, shape, and size.
   Ans: ndim is used to know number of dimensions. shape is used to know how many rows and coloumns.size used to know number of elements.
9. Predict the shape of a 3-row, 4-column array before running the code.

Answer: The shape is (3, 4), because the array has 3 rows and 4 columns.

5. Part C – Indexing

Use this array: marks = np.array([72, 85, 91, 66, 78, 95])

10. Print the first element and the last element.

```
import numpy as np
arr=np.array([11, 12, 13,26,78] )
print(arr[0])
print(arr[-1])
```

11. Print the element at index 2.
    Ans:print(marks[2])
12. Print the second-last element using negative indexing.
    Ans:print(marks[-2])
13. Change the value 66 to 70 and print the updated array.
    Ans:marks[3]=70
14. Use a 2D array [[11, 12, 13], [21, 22, 23]] to access 22 and 13.

```
import numpy as np
arr=np.array([[11, 12, 13], [21, 22, 23]] )
print(arr[0,2])
print(arr[1,1])
```

6. Part D – Slicing

Use this array: values = np.array([5, 10, 15, 20, 25, 30, 35, 40])

15. Select the first four elements.

```
import numpy as np
values = np.array([5, 10, 15, 20, 25, 30, 35, 40])
print(values[0:4])
```

16. Select the elements from index 2 up to, but not including, index 6.

Answer:
print(values[2:6]) # [15 20 25 30]

17. Select every second element.

Answer:
print(values[::2]) # [5 15 25 35]

```
import numpy as np
values = np.array([5, 10, 15, 20, 25, 30, 35, 40])
print(values[1:8:2])
```

18. Select the last three elements.

Answer:
print(values[-3:]) # [30 35 40]

19. For matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), select the first two rows and the last two columns.

Answer:
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[:2, -2:]) # [[2 3] [5 6]]

20. Write one sentence explaining why the stop index is not included in a slice.

Answer: Python slicing includes the start index but stops before the stop index, which makes the slice length easy to calculate.

7. Part E – Vectorized Operations

Use these arrays:

a = np.array([10, 20, 30, 40])

b = np.array([1, 2, 3, 4])

21. Calculate a + b, a - b, a * b, and a / b.

```
import numpy as np
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print(a+b)
print(abs(a-b))
print(a*b)
print(a/b)
```

22. Add 5 to every element of a without using a loop.

```
import numpy as np
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print(a+5)
print(b+5)
```

23. Multiply every element of a by 3.

```
import numpy as np
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print(a*3)
print(b*3)
```

24. Find the elements of a that are greater than 20 using a comparison operation.

Answer:
print(a[a > 20]) # [30 40]

25. Use a Boolean condition to select marks greater than or equal to 80 from np.array([55, 82, 91, 67, 88]).

Answer:

```
import numpy as np
marks = np.array([55, 82, 91, 67, 88])
print(marks[marks >= 80])  # [82 91 88]
```

In your own words, explain what vectorized operations do.

Answer: Vectorized operations apply an operation to many array elements at once without writing a separate loop for each element.

8. Part F – Debugging Practice

For each snippet, identify the problem and correct it.

Debugging Problem 1

```
import numpy as np
arr = np.array([10, 20, 30])
print(arr[3])
```

ans:the total elements in the list is 3 but we are accesing for 4th element.to get third element.

```
import numpy as np
arr = np.array([10, 20, 30])
print(arr[2])
```

in this indexing starts from 0.

Debugging Problem 2

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[1:10])
```

ans:this program will execute but there are only 4 elements in the array but we are accessing 10 elements.

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[1:4])
```

Debugging Problem 3

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20])
print(a + b)
```

ans:we cannot add both arrays because one array is containing two other containg three .

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20,10])
print(a + b)
```

Debugging Problem 4

```
import numpy as np
arr = np.array([[1, 2], [3, 4]])
print(arr[2, 0])
```

ans:we accessing for 3 array first element.but there is no third array .

```
import numpy as np
arr = np.array([[1, 2], [3, 4]])
print(arr[1, 0])
```

9. Part G – Main Hands-on Activity: Student Marks Analyzer

Create a NumPy-based program to analyze student marks. Use this data:

Complete program for Part G:

```
marks = np.array([[78, 85, 92], [66, 74, 81], [90, 88, 95], [55, 69, 72]])
print('Marks array:\n', marks)
print('Dimensions:', marks.ndim)
print('Shape:', marks.shape)
print('First student marks:', marks[0])
print('Second subject marks:', marks[:, 1])
adjusted = marks + 5
adjusted = np.minimum(adjusted, 100)
print('Adjusted marks (bonus capped at 100):\n', adjusted)
print('Average for each student:', adjusted.mean(axis=1))
print('Highest original mark:', marks.max())
print('Adjusted scores 80 or above:', adjusted[adjusted >= 80])
```

marks = np.array([[78, 85, 92], [66, 74, 81], [90, 88, 95], [55, 69, 72]])

Each row represents one student; each column represents a subject.

Your program should:

- Print the full marks array, its dimensions, and its shape.
- Print the marks of the first student.
- Print the marks for the second subject for all students.
- Add 5 bonus marks to every score using a vectorized operation.
- Cap adjusted marks at 100 using np.minimum or an equivalent NumPy operation.
- Calculate each student's average marks using an axis argument.
- Find the highest mark in the full array.
- Select all adjusted scores that are 80 or above.
- Print results with meaningful labels.

10. Part H – Explain in Your Own Words

1. What is NumPy, and why is it useful?

Answer: NumPy is a Python library for working with numerical data.

2. What is a NumPy array?

Answer: A NumPy array is a collection of elements, usually of the same data type, arranged in one or more dimensions.

3. What is the difference between a Python list and a NumPy array?

Answer: A Python list can store different types of objects and is general-purpose. A NumPy array is designed for numerical work, usually stores one data type, and supports fast vectorized calculations.

4. What do ndim, shape, size, and dtype represent?

Answer: ndim gives the number of dimensions; shape gives the size along each dimension; size gives the total number of elements; dtype gives the elements' data type.

5. What is indexing? What is negative indexing?

Answer: Indexing accesses a particular element by its position. Negative indexing counts from the end, so -1 means the last element.

6. What is slicing?

Answer: Slicing selects a portion of an array using start, stop, and optional step values. The stop index is not included.

7. What is a vectorized operation?

Answer: A vectorized operation performs the same operation on multiple array elements at once without needing an explicit loop.

8. Why must arrays generally have compatible shapes for element-wise arithmetic?

Answer: Element-wise arithmetic pairs elements by position. Compatible shapes ensure the arrays can be paired correctly or broadcast according to NumPy's rules.

11. Daily Deliverable

Submit your working Python script: numpy_assignment_YourName.py

Your script should demonstrate:

- Array creation and dtype
- 1D, 2D, and 3D dimensions
- ndim, shape, and size
- Positive and negative indexing
- 1D and 2D slicing
- Vectorized arithmetic and comparisons
- A complete marks-analysis activity
- Readable code and clear output

12. Student Submission Checklist

☐ I can create NumPy arrays.

☐ I can identify 1D, 2D, and 3D arrays.

☐ I understand ndim, shape, size, and dtype.

☐ I can access elements using indexing.

☐ I can extract data using slicing.

☐ I can perform arithmetic without element-by-element loops.

☐ I can filter array values using Boolean conditions.

☐ I tested and corrected the debugging problems.

☐ I can explain the main concepts in my own words.

13. After This Assignment, You Should Be Able To

- Create and inspect NumPy arrays.
- Navigate 1D, 2D, and 3D array data.
- Select elements and subarrays using indexing and slicing.
- Process numerical data using vectorized operations.
- Use NumPy to perform a small practical data-analysis task.

Key Principle: Use NumPy to organize numerical data and process many values efficiently.